

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN



HCMUTE

MÔN HỌC: XỬ LÝ ẢNH SỐ

ĐỀ TÀI CUỐI KỲ:

**XÂY DỰNG HỆ THỐNG CẢNH BÁO TẾ NGÃ TỰ ĐỘNG
CHO NGƯỜI CAO TUỔI**

GVHD: PGS.TS. Hoàng Văn Dũng

Mã LHP: DIPR430685_06CLC

Học kỳ: II

Năm học: 2025 – 2026

Danh sách sinh viên thực hiện: Nhóm 05

MSSV	Họ tên
23110078	Nguyễn Thái Bảo
23110110	Lê Quang Hưng
23110111	Lương Nguyễn Thành Hưng

Thành phố Hồ Chí Minh, tháng 06 năm 2026

MỤC LỤC

DANH MỤC HÌNH ẢNH.....	5
PHẦN 1: MỞ ĐẦU	1
1.1. Lý do chọn đề tài.....	1
1.2. Mục tiêu nghiên cứu	2
1.3. Đối tượng và phạm vi nghiên cứu:	2
1.3.1. Đối tượng nghiên cứu	2
1.3.2. Phạm vi nghiên cứu	3
PHẦN 2: CƠ SỞ LÝ THUYẾT.....	4
2.1. Tổng quan về xử lý ảnh số và tiền xử lý	4
2.1.1. Kỹ thuật lọc và khử nhiễu trong miền không gian	4
2.1.2. Cân bằng sáng thích nghi (CLAHE).....	4
2.2. Mô hình YOLO-POSE cho ước lượng tư thế người.....	4
2.2.1. Tổng quan bài toán Pose Estimation	4
2.2.2. Kiến trúc YOLOv11s-Pose.....	5
2.3. Mạng hồi quy hai chiều PoseBiGRU-Attention.....	5
2.3.1. Tầm quan trọng của phân tích chuỗi thời gian	5
2.3.2. Kiến trúc mô hình	5
2.4. Cơ chế cửa sổ trượt (Sliding Window).....	6
PHẦN 3: PHƯƠNG PHÁP VÀ THIẾT KẾ HỆ THỐNG.....	8
3.1. Kiến trúc tổng quát của hệ thống.....	8
3.2. Module thu nhận và tiền xử lý video (DIP Preprocessing)	9
3.2.1. Thiết kế module CameraManager	9
3.2.2. Pipeline tiền xử lý thích nghi (Adaptive Enhancement)	9
3.3. Module nhận dạng và theo dõi đối tượng.....	10
3.3.1. YOLO-Pose Inference	10
3.3.2. Bộ lọc 3 tầng xác minh người thật.....	10

3.4. Module phân loại hành vi té ngã	11
3.4.1. Tầng 1 - Mạng PoseBiGRU-Attention (Model-based)	11
3.4.2. Tầng 2 - Hệ thống quy tắc Heuristic (Rule-based Voting).....	11
3.4.3. Các biện pháp chống cảnh báo giả (False Positive Mitigation)	12
3.4.4. Cơ chế Debounce/Cooldown chống Spam Alert.....	12
3.5. Module ghi log và lưu ảnh bằng chứng (Snapshot)	13
3.6. Module cảnh báo qua Telegram	13
PHẦN 4: CÀI ĐẶT VÀ TRIỂN KHAI HỆ THỐNG	15
4.1. Môi trường phát triển và công nghệ sử dụng.....	15
4.1.1. Môi trường phần cứng	15
4.1.2. Công nghệ phần mềm	15
4.2. Xây dựng giao diện người dùng (GUI)	16
4.2.1. Kiến trúc MVC đơn giản	16
4.2.2. Các màn hình chính	16
4.2.3. Thanh menu bên trái (Sidebar)	19
4.3. Huấn luyện mô hình PoseBiGRU	20
4.3.1. Bộ dữ liệu.....	20
4.3.2. Quy trình chuẩn bị dữ liệu (Data Pipeline).....	20
4.3.3. Siêu tham số huấn luyện	20
4.3.4. Môi trường huấn luyện	21
4.4. Kết nối Camera IP từ điện thoại	22
PHẦN 5: THỬ NGHIỆM VÀ ĐÁNH GIÁ	23
5.1. Các độ đo đánh giá hiệu năng.....	23
5.1.1. Đánh giá năng lực phân loại	23
5.1.2. Đánh giá hiệu năng hệ thống	23
5.2. Kịch bản thử nghiệm	23

5.2.1. Kết quả Kịch bản Positive - Té ngã thực tế	23
5.2.2. Kết quả Kịch bản Hard-Negative - Hành vi gây nhiễu	25
5.3. Kết quả huấn luyện mô hình PoseBiGRU	25
5.3.1. Đường cong Loss	25
5.3.2. Kết quả đánh giá trên tập Validation	26
5.3.3. Ma trận nhầm lẫn (Confusion Matrix).....	27
5.5. Phân tích và thảo luận	28
5.5.1. Điểm mạnh.....	28
5.5.2. Hạn chế và hướng cải thiện	28
PHẦN 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	30
6.1. Kết luận.....	30
6.2. Hướng phát triển.....	30

DANH MỤC HÌNH ẢNH

Hình 2. 1. Sơ đồ giải thích model	6
Hình 3. 1. Sơ đồ kiến trúc tổng quát hệ thống	8
Hình 3. 2. So sánh frame trước và sau khi qua module DIP Enhancement	10
Hình 3. 3. Ảnh chụp tin nhắn cảnh báo té ngã trên ứng dụng Telegram..	14
Hình 4. 1. Ảnh chụp màn hình Đăng nhập	16
Hình 4. 2, Ảnh chụp màn hình Giám sát Camera khi hệ thống an toàn ...	17
Hình 4. 3. Ảnh chụp màn hình Giám sát Camera khi phát hiện té ngã (khung đỏ)	17
Hình 4. 4. Ảnh chụp màn hình Cài đặt hệ thống 1	18
Hình 4. 5. Ảnh chụp màn hình Cài đặt hệ thống 2	18
Hình 4. 6. Ảnh chụp màn hình Lịch sử cảnh báo té ngã với ảnh Snapshot	19
Hình 4. 7. Ảnh chụp Kaggle Notebook đang train	21
Hình 4. 8. Ảnh chụp điện thoại đang chạy IP Camera và Laptop nhận stream	22
Hình 5. . Đường cong Training Loss và Validation Loss.....	26
Hình 5. . Ma trận nhầm lẫn tại ngưỡng tối ưu 0.73	27

PHẦN 1: MỞ ĐẦU

1.1. Lý do chọn đề tài

Hiện nay, sự già hóa dân số đang đặt ra những thách thức to lớn cho toàn cầu trong việc xây dựng một môi trường sống độc lập và an toàn cho người cao tuổi. Trong đó, té ngã được ghi nhận là một trong những rủi ro sức khỏe nghiêm trọng nhất. Theo báo cáo từ Cẩm nang Y khoa MSD (MSD Manuals), ở những người từ 65 tuổi trở lên, té ngã là nguyên nhân hàng đầu gây tử vong do chấn thương và là nguyên nhân tử vong đứng thứ bảy nói chung [1]. Người cao tuổi có nguy cơ vấp ngã rất cao do sự suy giảm chức năng vận động, thị lực kém và các bệnh lý nền đi kèm [2]. Nghiên cứu cũng chỉ ra rằng, việc phát hiện chậm trễ và thiếu can thiệp y tế kịp thời sau khi ngã sẽ làm tăng đột biến tỷ lệ mắc các biến chứng nghiêm trọng như gãy xương, chấn thương sọ não, thậm chí dẫn đến tử vong. Do đó, việc thiết lập một hệ thống giám sát nhằm phát hiện và cảnh báo tức thời sự cố té ngã là yêu cầu vô cùng cấp thiết.

Để giải quyết bài toán này, nhiều hệ thống phát hiện té ngã (Fall Detection Systems) đã được nghiên cứu và phát triển. Phổ biến nhất là các thiết bị đeo trên người (wearable sensors) như đồng hồ thông minh hoặc vòng cổ tích hợp gia tốc kế. Tuy nhiên, các thiết bị này tồn tại nhược điểm chí mạng trong thực tiễn: tính hiệu quả của chúng phụ thuộc hoàn toàn vào việc người dùng có mang thiết bị liên tục hay không. Việc người cao tuổi thường xuyên quên đeo thiết bị, quên sạc pin, hoặc cảm thấy vướng víu đã làm giảm đáng kể độ tin cậy của hệ thống [3]. Bên cạnh đó, các hệ thống cảm biến gắn trên sàn nhà (smart flooring) dù khắc phục được nhược điểm của thiết bị đeo nhưng lại có chi phí lắp đặt, bảo trì rất đắt đỏ và khó triển khai trên diện rộng [3].

Đứng trước những hạn chế đó, các phương pháp tiếp cận không xâm nhập (unobtrusive methods) dựa trên camera giám sát và kỹ thuật Xử lý ảnh số (Digital Image Processing) đang nổi lên như một giải pháp tối ưu. Việc ứng dụng các thuật toán xử lý ảnh - từ các bộ lọc trong miền không gian (Spatial Domain Filters) để tiền xử lý và khử nhiễu, đến các kỹ thuật trích xuất đặc trưng (Feature Extraction) và nhận dạng tư thế người [4] - cho phép máy tính theo dõi và phân tích hành vi của con người một cách liên tục, hoàn toàn tự động mà không đòi hỏi người cao tuổi phải mang vác bất kỳ thiết bị nào trên cơ thể. Sự kết hợp giữa xử lý ảnh truyền thống và các mô hình học sâu hiện đại có khả

năng phân biệt chính xác hành vi té ngã với các hoạt động sinh hoạt bình thường, đảm bảo tính thời gian thực (real-time) và giảm thiểu cảnh báo giả [5].

Nhận thấy rõ tính cấp thiết từ thực tiễn xã hội cùng với tiềm năng to lớn của công nghệ, nhóm quyết định lựa chọn đề tài: **Xây dựng hệ thống cảnh báo té ngã tự động cho người cao tuổi.**

1.2. Mục tiêu nghiên cứu

Mục tiêu tổng quan của đề tài là nghiên cứu, thiết kế và triển khai một hệ thống giám sát tự động không xâm nhập dựa trên công nghệ Thị giác máy tính (Computer Vision) và Xử lý ảnh số. Hệ thống nhằm mục đích theo dõi, phát hiện sớm và đưa ra cảnh báo theo thời gian thực đối với hành vi té ngã của người cao tuổi trong môi trường trong nhà (indoor environment).

Các mục tiêu kỹ thuật cụ thể bao gồm:

Về thu nhận và tiền xử lý dữ liệu: Áp dụng các kỹ thuật xử lý ảnh trong miền không gian nhằm khử nhiễu và cân bằng độ sáng, độ tương phản cho chuỗi video đầu vào. Module tiền xử lý phải hoạt động thích nghi (adaptive) với mọi điều kiện ánh sáng.

Về nhận dạng và theo dõi đối tượng: Tích hợp mô hình YOLO-Pose để nhận diện người và trích xuất liên tục các đặc trưng khung xương (Skeletal Keypoints) qua từng khung hình. Kết hợp bộ theo dõi ByteTrack để gán và duy trì định danh (ID) cho mỗi đối tượng xuyên suốt video.

Về thuật toán phát hiện té ngã: Xây dựng kiến trúc phát hiện hai tầng (Two-tier Detection): (1) Mạng PoseBiGRU-Attention do nhóm tự huấn luyện để phân loại chuỗi tư thế, kết hợp (2) Hệ thống quy tắc heuristic dựa trên hình học để tăng cường tốc độ phản ứng và xử lý các trường hợp biên. Hệ thống phải phân biệt được hành vi té ngã thật với các hoạt động gây nhiễu như cúi nhặt đồ, ngồi thụp xuống, hay quỳ gối.

Về hệ thống cảnh báo: Khi phát hiện té ngã, hệ thống phát âm thanh cảnh báo, hiển thị khung đỏ trên màn hình, lưu ảnh bằng chứng (Snapshot), ghi lịch sử CSV, và gửi cảnh báo qua Telegram kèm ảnh chụp đến người giám hộ.

1.3. Đối tượng và phạm vi nghiên cứu:

1.3.1. Đối tượng nghiên cứu

Đối tượng giám sát: Hành vi và tư thế động học của con người trong không gian sinh hoạt. Trọng tâm là phân biệt giữa sự cố “té ngã vô thức” và các hoạt động sinh hoạt thường ngày (ADLs).

Đối tượng xử lý kỹ thuật: Các khung hình tĩnh và chuỗi khung hình liên tục được trích xuất từ nguồn video kỹ thuật số, cụ thể là các đặc trưng bộ khung xương (Skeletal Keypoints) của con người trên không gian ảnh 2D.

1.3.2. Phạm vi nghiên cứu

Về môi trường: Hệ thống tập trung hoạt động trong không gian kín (indoor), tạm thời bỏ qua bối cảnh ngoại cảnh đông người.

Về thiết bị: Hệ thống tiếp nhận nguồn dữ liệu hình ảnh quang học (RGB) từ camera tĩnh đơn tròng hoặc Camera IP (qua giao thức HTTP/RTSP). Loại trừ hoàn toàn wearable sensors.

Về giải pháp phần mềm: Sử dụng các kỹ thuật DIP cơ bản ở giai đoạn tiền xử lý, mô hình YOLO-Pose đã được huấn luyện trước (pre-trained) để phát hiện người, và mạng PoseBiGRU tự huấn luyện cho bài toán phân loại chuỗi hành vi.

PHẦN 2: CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về xử lý ảnh số và tiền xử lý

Tiền xử lý ảnh (Preprocessing) là giai đoạn nền tảng đối với hiệu năng của bất kỳ hệ thống thị giác máy tính nào. Quy trình xử lý ảnh cơ bản bao gồm: Thu nhận ảnh (Image Acquisition), Nâng cao chất lượng ảnh (Image Enhancement), Phân vùng ảnh (Segmentation) và Trích xuất đặc trưng (Feature Extraction) [6].

2.1.1. Kỹ thuật lọc và khử nhiễu trong miền không gian

Kỹ thuật xử lý trong miền không gian (Spatial Domain) can thiệp trực tiếp giá trị của từng pixel trên ma trận ảnh, biểu diễn qua $g(x, y) = T[f(x, y)]$ [7]. Bộ lọc Gaussian được sử dụng trong hệ thống, hoạt động dựa trên nguyên lý tính trung bình có trọng số của các pixel lân cận, giúp khử nhiễu tần số cao mà vẫn bảo toàn được đường biên cấu trúc cơ thể người - điều kiện tiên quyết để YOLO nội suy chính xác các keypoints.

2.1.2. Cân bằng sáng thích nghi (CLAHE)

Trong môi trường nhà ở, ánh sáng thường bất ổn (ngược sáng cửa sổ, góc khuất tạo bóng, ánh sáng yếu ban đêm). Kỹ thuật CLAHE (Contrast Limited Adaptive Histogram Equalization) được áp dụng - là phiên bản cải tiến của Histogram Equalization, chia ảnh thành các ô nhỏ (tiles) và cân bằng histogram cục bộ trên từng ô, đồng thời giới hạn biên độ tăng cường (clip limit) để tránh khuếch đại nhiễu. Phương pháp này giúp khôi phục chi tiết vùng tối mà không làm quá sáng các vùng đã đủ sáng [7].

2.2. Mô hình YOLO-POSE cho ước lượng tư thế người

2.2.1. Tổng quan bài toán Pose Estimation

Ước lượng tư thế người (Human Pose Estimation) là bài toán xác định vị trí các điểm mốc giải phẫu (keypoints) trên cơ thể từ ảnh 2D. Hệ thống sử dụng tiêu chuẩn COCO-17 Keypoints gồm 17 điểm: mũi, mắt (trái/phải), tai (trái/phải), vai (trái/phải), khuỷu tay (trái/phải), cổ tay (trái/phải), hông (trái/phải), đầu gối (trái/phải), mắt cá chân (trái/phải). Mỗi keypoint được biểu diễn bởi bộ ba (x_i, y_i, c_i) - tọa độ 2D và độ tin cậy.

2.2.2. Kiến trúc YOLOv11s-Pose

Hệ thống sử dụng mô hình YOLOv11s-Pose (bản Small) thay vì bản Nano, vì bản Small có khả năng nhận diện người ở tư thế nằm chính xác hơn do số lượng tham số lớn hơn. Mô hình này thực hiện đồng thời hai tác vụ: (1) Phát hiện người (Object Detection) bằng bounding box, và (2) Ước lượng tọa độ 17 keypoints cho mỗi người phát hiện được. Bộ trọng số sử dụng đã được huấn luyện trước trên tập COCO và được đóng băng (freeze) - không cần huấn luyện lại.

Kết hợp với thuật toán theo dõi ByteTrack, hệ thống duy trì định danh (Track ID) cho mỗi người xuyên suốt video, cho phép phân tích trạng thái tư thế theo thời gian cho từng cá nhân riêng biệt.

2.3. Mạng hồi quy hai chiều PoseBiGRU-Attention

2.3.1. Tầm quan trọng của phân tích chuỗi thời gian

Hành vi té ngã là một quá trình biến thiên liên tục - không thể phán đoán chính xác chỉ từ một khung hình tĩnh duy nhất. Một người nằm trên sàn có thể là do đã ngã, hoặc đơn giản là đang nằm nghỉ. Sự khác biệt nằm ở quỹ đạo chuyển động trước đó: ngã thật sẽ kèm theo sự sụp đổ nhanh của trọng tâm cơ thể, trong khi nằm có chủ ý diễn ra từ từ và có kiểm soát. Do đó, việc sử dụng mạng hồi quy (RNN) để phân tích chuỗi tư thế theo thời gian là cần thiết.

2.3.2. Kiến trúc mô hình

Mô hình PoseBiGRU-Attention do nhóm nghiên cứu tự thiết kế và huấn luyện, bao gồm 5 khối chức năng chính:

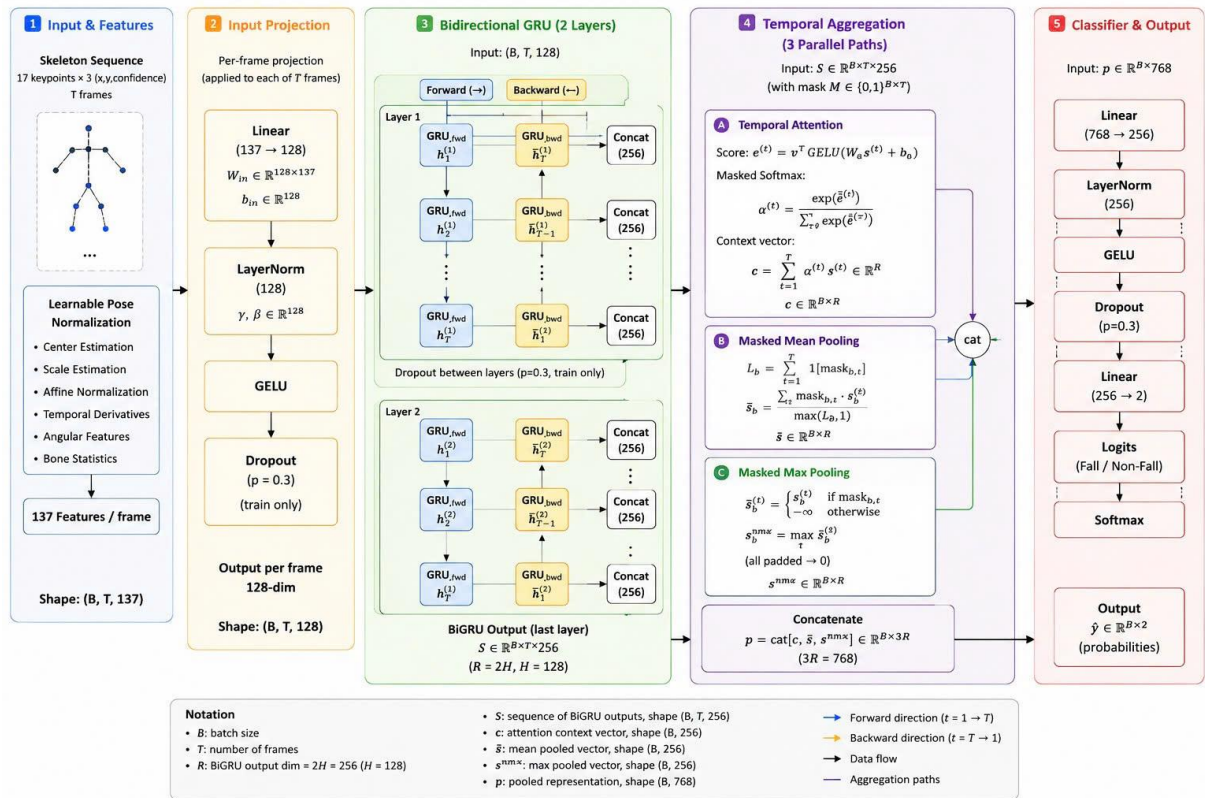
Khối 1 - Input & Features: Nhận đầu vào là chuỗi Skeleton Sequence có kích thước $(B, T, 17, 3)$ - gồm B batch, T frames, 17 keypoints, mỗi keypoint có 3 giá trị $(x, y, confidence)$. Module Learnable Pose Normalization thực hiện chuẩn hóa tọa độ (dựa trên trung tâm cơ thể, tỉ lệ thân), tính toán vận tốc và gia tốc từng khớp, trích xuất đặc trưng góc xương, thống kê độ dài xương - tổng cộng cho ra 137 đặc trưng/frame.

Khối 2 - Input Projection: Chiếu 137 đặc trưng thô vào không gian biểu diễn 128 chiều bằng lớp Linear, kết hợp LayerNorm, hàm kích hoạt GELU và Dropout ($p=0.3$).

Khối 3 - Bidirectional GRU (2 lớp): Chuỗi 128 chiều đi qua 2 lớp GRU hai chiều (forward + backward), mỗi hướng có hidden_size = 128, cho đầu ra mỗi frame là vector 256 chiều. GRU hai chiều cho phép mô hình “nhìn” cả quá khứ và tương lai trong chuỗi - đặc biệt quan trọng khi giai đoạn té ngã nằm ở giữa clip.

Khối 4 - Temporal Aggregation (3 nhánh song song): Đầu ra chuỗi GRU $(B, T, 256)$ được tổng hợp đồng thời qua: (A) Temporal Attention - học trọng số attention $\alpha^{(t)}$ cho mỗi frame, tạo context vector; (B) Masked Mean Pooling - trung bình có trọng số các frame hợp lệ; (C) Masked Max Pooling - lấy giá trị cực đại. Ba vector 256 chiều được nối (concatenate) thành vector biểu diễn 768 chiều.

Khối 5 - Classifier & Output: Vector 768 chiều đi qua lớp Linear $\rightarrow$ LayerNorm $\rightarrow$ GELU $\rightarrow$ Dropout $\rightarrow$ Linear $(256 \rightarrow 2) \rightarrow$ Softmax, cho ra xác suất 2 lớp: Fall và Non-Fall.



Hình 2. 1. Sơ đồ giải thích model

2.4. Cơ chế cửa sổ trượt (Sliding Window)

Chuỗi tư thế từ YOLO-Pose liên tục theo thời gian, nhưng mô hình PoseBiGRU nhận đầu vào có độ dài cố định T frames. Thuật toán cửa sổ trượt (Sliding Window) được sử dụng: mỗi khi thu thập đủ T frame tư thế mới nhất của một người (track ID), hệ thống đóng gói thành một chuỗi $(1, T, 17, 3)$ và đưa qua mô hình. Cửa sổ trượt theo kiểu FIFO - frame cũ nhất bị loại bỏ khi frame mới nhất được thêm vào - đảm bảo phân tích luôn phản ánh trạng thái hiện tại nhất.

PHẦN 3: PHƯƠNG PHÁP VÀ THIẾT KẾ HỆ THỐNG

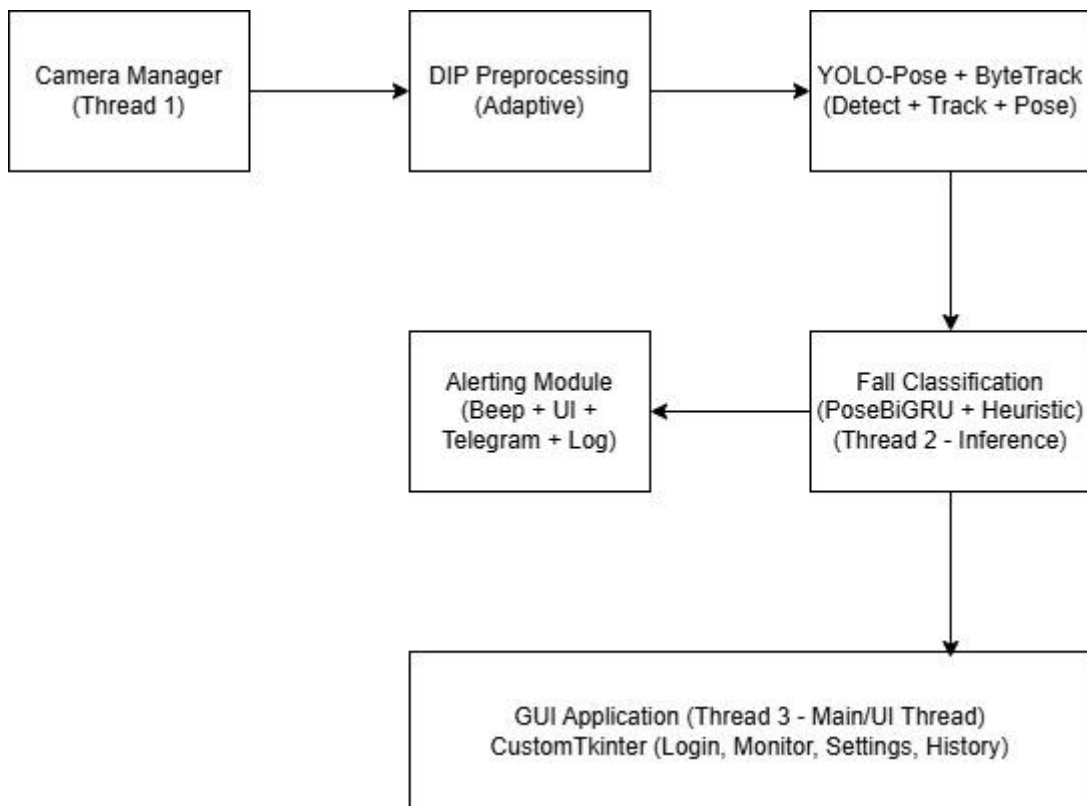
3.1. Kiến trúc tổng quát của hệ thống

Hệ thống được thiết kế theo mô hình pipeline đa luồng (Multi-threaded Pipeline), gồm 6 giai đoạn xử lý nối tiếp, được phân chia thành 3 luồng độc lập để đảm bảo hiệu năng thời gian thực:

Luồng 1 - Camera Thread (“CameraManager”): Đọc frame liên tục từ nguồn video (Webcam, file video, hoặc Camera IP qua HTTP/RTSP), áp dụng bộ lọc DIP thích nghi, chuyển đổi BGR→RGB, và đẩy vào buffer chia sẻ (shared latest frame).

Luồng 2 - Inference Thread (“InferenceManager” + “FallDetectorWorker”): Lấy frame mới nhất từ buffer, chạy qua YOLO-Pose → trích xuất skeleton → đẩy vào sliding window → PoseBiGRU inference → kết hợp heuristic → cập nhật trạng thái fall_state cho UI đọc.

Luồng 3 - UI Thread (“MainApp”): Hiển thị giao diện, đọc trạng thái từ Inference Thread mỗi 500ms, kích hoạt cảnh báo khi phát hiện ngã mới.



Hình 3. 1. Sơ đồ kiến trúc tổng quát hệ thống

3.2. Module thu nhận và tiền xử lý video (DIP Preprocessing)

3.2.1. Thiết kế module *CameraManager*

Class **CameraManager** đóng vai trò lớp trừu tượng (Abstraction Layer) cho mọi nguồn video đầu vào. Nhờ thiết kế này, hệ thống có thể chuyển đổi linh hoạt giữa:

- Webcam: Tham số 'source=0' (mặc định)
- File video: Tham số 'source="path/to/video.mp4"' - tự động lặp lại khi hết video
- Camera IP: Tham số 'source=<http://192.168.1.x:8080/video>' - hỗ trợ HTTP stream và RTSP

CameraManager chạy trên luồng riêng, đọc frame theo tốc độ FPS gốc của nguồn video, áp dụng pipeline DIP trên từng frame rồi lưu vào biến chia sẻ ("**latest_frame**") được bảo vệ bằng ``threading.Lock``.

3.2.2. Pipeline tiền xử lý thích nghi (*Adaptive Enhancement*)

Thay vì áp dụng bộ lọc cố định, hệ thống triển khai một pipeline tự hiệu chuẩn (Auto-Calibration) gồm các bước:

1. Giai đoạn hiệu chuẩn (Calibration Phase) - Khi camera bắt đầu hoạt động, hệ thống thu thập 30 frame đầu tiên để tính toán các chỉ số baseline:

- Baseline Brightness: Độ sáng trung bình của ảnh.
- Baseline Noise: Mức nhiễu ước tính (dựa trên độ lệch chuẩn của ảnh Laplacian).
- Baseline Contrast: Độ tương phản trung bình.

2. Giai đoạn xử lý thích nghi (Adaptive Processing): Dựa trên so sánh giữa frame hiện tại và baseline:

- Khử nhiễu (Denoising): Áp dụng 'cv2.fastNlMeansDenoisingColored()' khi noise level vượt ngưỡng, với cường độ lọc tỷ lệ thuận với mức nhiễu. Gaussian Blur (3×3) được áp dụng bổ sung nếu nhiễu nặng.

- Cân bằng sáng (CLAHE): Áp dụng trên kênh L của không gian màu LAB khi độ sáng frame thấp hơn 70% baseline. Tham số 'clipLimit' và 'tileGridSize' được điều chỉnh tự động theo mức chênh lệch.

- Xử lý ngược sáng (Backlight Compensation): Phát hiện tình huống ngược sáng dựa trên histogram bi-modal, sau đó áp dụng tone mapping cục bộ để khôi phục chi tiết vùng tối.
- Cân bằng trắng (White Balance): Hiệu chỉnh màu sắc bằng thuật toán Grey World khi phát hiện dịch chuyển màu (color cast).



Hình 3. 2. So sánh frame trước và sau khi qua module DIP Enhancement

3.3. Module nhận dạng và theo dõi đối tượng

3.3.1. YOLO-Pose Inference

Class **FallDetectorWorker** tích hợp mô hình **yolo11s-pose.pt** với cấu hình inference được tối ưu:

Tham số	Giá trị	Lý do
conf	0.10	Ngưỡng rất thấp để không bỏ sót người ở tư thế nằm (box nhỏ, mờ).
iou	0.45	Ngưỡng NMS chuẩn cho single-class detection.
imgsz	960	Độ phân giải cao hơn mặc định (640) để tăng độ chính xác key points.
classes	[0]	Chỉ phát hiện class “person”
tracker	bytetrack.yaml	ByteTrack - không dùng GMC optional flow, tránh crash khi frame size thay đổi

3.3.2. Bộ lọc 3 tầng xác minh người thật

Do YOLO với “conf=0.10” sẽ phát hiện được nhiều box - bao gồm cả ghế, vật dụng bị nhầm lẫn - hệ thống triển khai bộ lọc 3 tầng:

- Tầng 1 - YOLO Box Confidence: Loại bỏ các detection có confidence < 0.25
- Tầng 2 - Số keypoints nhìn thấy: Yêu cầu tối thiểu 3 keypoints có confidence > 0.3 (ngưỡng thấp để không loại người nằm bị che)
- Tầng 3 - Core Body Parts: Yêu cầu ít nhất 1 vai hoặc 1 hông phải nhìn thấy - đây là các điểm mốc cốt lõi xác định đó là người thật, không phải đồ vật

3.4. Module phân loại hành vi té ngã

Hệ thống sử dụng kiến trúc Two-tier Detection - kết hợp mô hình học sâu PoseBiGRU và hệ thống quy tắc heuristic.

3.4.1. Tầng 1 - Mạng PoseBiGRU-Attention (Model-based)

Mỗi khi thu thập đủ chuỗi 30 frame tư thế cho một Track ID, chuỗi được đóng gói thành tensor (1,30,17,3) và đưa qua mô hình PoseBiGRU. Đầu ra Softmax cho xác suất 2 lớp là Fall và Non-Fall.

- Nếu $Fall \geq \text{“conf_threshold”}$ (mặc định 0.50, có thể tùy chỉnh lại trên UI dựa trên kết quả tốt nhất của model): hệ thống kích hoạt cảnh báo.

- Nếu $Fall \geq \text{“rule_model_soft_threshold”}$ (0.25): trạng thái “khả nghi” được đánh dấu, cung cấp thêm điểm cho cơ chế voting ở tầng 2.

3.4.2. Tầng 2 - Hệ thống quy tắc Heuristic (Rule-based Voting)

Song song với mô hình AI, hệ thống tính toán 8 chỉ số hình học và động học từ dữ liệu YOLO-Pose tại mỗi frame:

Chỉ số	Mô tả	Điểm
lying_by_box	Tỷ lệ chiều rộng/chiều cao bounding box ≥ 1.35	+2
torso_horizontal	Góc giữa vai và hông so với phương ngang $\leq 35^\circ$	+1
head_below_hip	Đầu nằm thấp hơn hông (trục Y hướng xuống)	+2
collapsed	Chiều cao box hiện tại < 55% chiều cao cực đại đã ghi nhận	+1
height_ratio_drop	Chiều cao box giảm xuống dưới 50% so với max	+1

sudden_motion	Tốc độ di chuyển trọng tâm $\geq 70\%$ chiều cao frame/giây	+2
lower_body_area	Đối tượng nằm ở nửa dưới khung hình	+1
model_suspicious	PoseBiGRU cho Fall ≥ 0.25	+2

Cơ chế biểu quyết (Voting): Tổng điểm $\geq 4.5 \rightarrow$ trạng thái “khả nghi”. Khi “khả nghi” liên tục ≥ 2 frame và thỏa thêm một trong ba điều kiện: (1) có dáng nằm rõ ràng, (2) sập đồ đột ngột, hoặc (3) Model AI tin cậy cao - hệ thống kích hoạt cảnh báo.

3.4.3. Các biện pháp chống cảnh báo giả (False Positive Mitigation)

Trong quá trình phát triển, nhóm đã xử lý các trường hợp thực tế gây cảnh báo giả:

- Cúi nhặt đồ / Quỳ gối: Tăng ngưỡng “lying_by_box” từ 1.1 $\rightarrow$ 1.35; chỉ cho “torso_horizontal” 1 điểm (thay vì 2); thêm chỉ số “head_below_hip” (+2 điểm) vì khi cúi nhặt đồ, đầu hiếm khi thấp hơn hông rõ rệt.

- Ngồi gần camera: Khi không phát hiện được phần dưới cơ thể (“lower_body_kpts == 0”) và box bị cắt ở mép dưới frame $\rightarrow$ tắt “lying_by_box”.

- Người đang đứng thẳng: Khi góc thân $\geq 60^\circ$ (torso_vertical) $\rightarrow$ bắt buộc “lying_by_box = False”.

- Người biến mất khỏi camera: Module “_check_lost_tracks” - khi một người đang ở trạng thái khả nghi hoặc đang đứng thẳng đột ngột mất track trong 0.2–2.0 giây $\rightarrow$ đánh dấu “LOST_AFTER_SUSPECTED_FALL”.

3.4.4. Cơ chế Debounce/Cooldown chống Spam Alert

Để tránh gửi cảnh báo lặp lại cho cùng một sự kiện ngã, hệ thống áp dụng cơ chế Cooldown trong “InferenceManager”:

1. Lần đầu phát hiện ngã: Gửi duy nhất 1 cảnh báo, lưu Track ID vào bảng cooldown.

2. Trong khi nạn nhân còn nằm: Liên tục cập nhật timestamp trong bảng cooldown và không gửi thêm cảnh báo mới.

3. Khi đứng lên: Track ID chuyển sang trạng thái “is_fall = False”. Chỉ khi duy trì trạng thái an toàn > “cooldown_time” (mặc định 3 giây) thì Track ID mới được xóa khỏi bảng cooldown - cho phép phát hiện lần ngã tiếp theo.

3.5. Module ghi log và lưu ảnh bằng chứng (Snapshot)

Class **FallLogger** hoạt động trên luồng nền riêng biệt, sử dụng hàng đợi Queue để tránh block UI:

1. Khi có sự kiện ngã mới, “log_fall()” được gọi kèm theo frame hiện tại (ảnh annotated từ YOLO).

2. Frame được chuyển từ RGB → BGR và lưu thành file JPEG tại “logs/snapshots/fall_cam_YYYYMMDD_HHMMSS_idXX.jpg”.

3. Dòng dữ liệu “[Timestamp, Camera_Source, Track_ID, Status, Snapshot_Path]” được đẩy vào queue.

4. Worker thread lấy dữ liệu từ queue và ghi vào file “fall_history.csv” theo mode append.

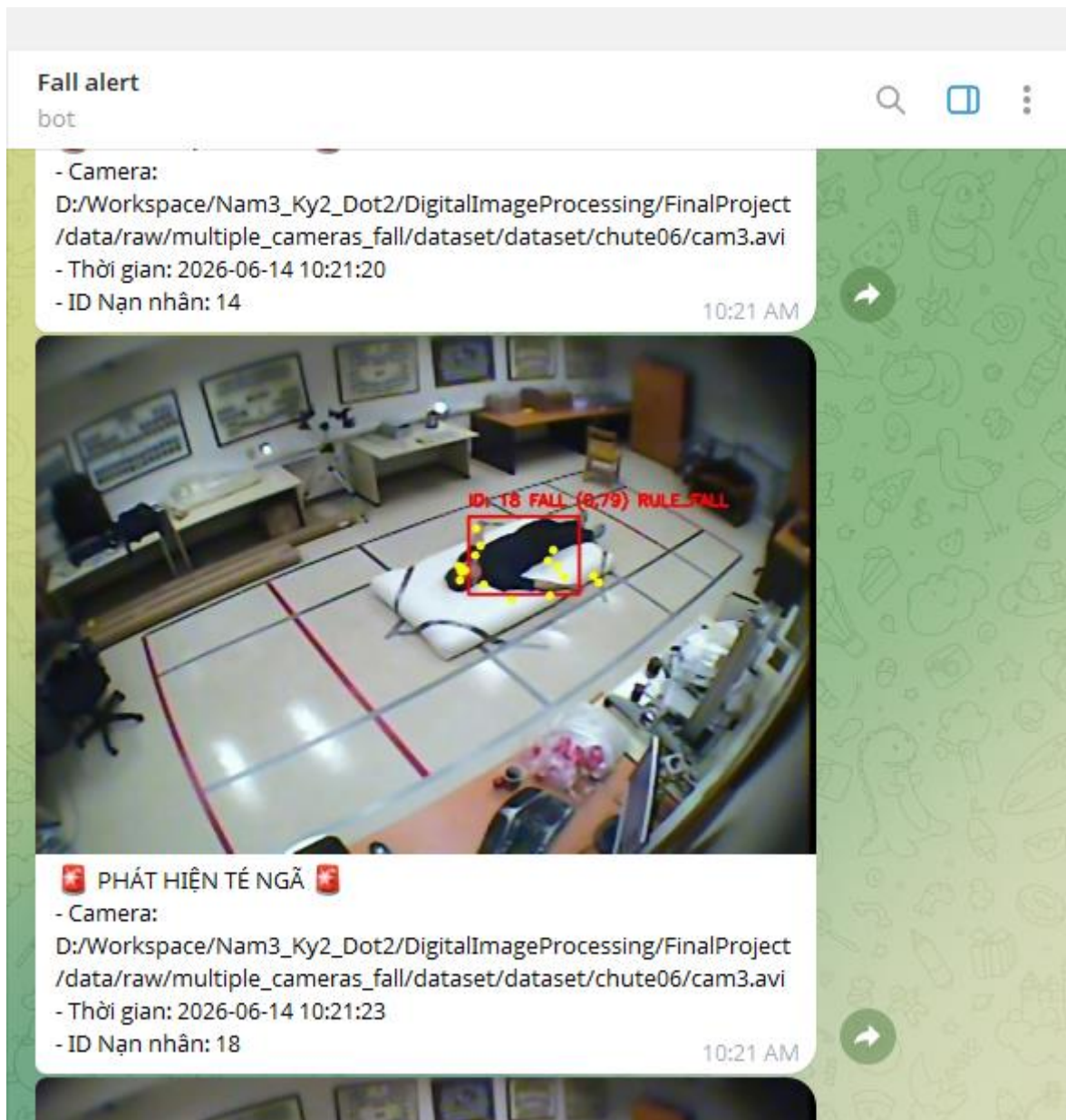
3.6. Module cảnh báo qua Telegram

Class **TelegramNotifier** cho phép gửi cảnh báo đến điện thoại người giám hộ qua Telegram Bot API:

Cấu hình: Bot Token và Chat ID được lưu trong file “telegram_config.json”, có thể bật/tắt và chỉnh sửa trực tiếp từ giao diện Cài đặt.

Khi phát hiện ngã: Hệ thống gửi HTTP POST đến Telegram API kèm theo: tin nhắn cảnh báo (chứa tên camera, thời gian, ID nạn nhân) và ảnh Snapshot chụp tại khoảnh khắc ngã.

Non-blocking: Việc gửi tin nhắn chạy trên thread daemon riêng, đảm bảo không ảnh hưởng đến tốc độ inference hay UI.



Hình 3. 3. Ảnh chụp tin nhắn cảnh báo té ngã trên ứng dụng Telegram

PHẦN 4: CÀI ĐẶT VÀ TRIỂN KHAI HỆ THỐNG

4.1. Môi trường phát triển và công nghệ sử dụng

4.1.1. Môi trường phần cứng

Thành phần	Cấu hình
Máy phát triển (Dev)	CPU Intel/AMD, RAM $\geq 8GB$, HDD/SSD
Môi trường huấn luyện (Train)	Kaggle Notebook - GPU NVIDIA Tesla T4 x 2
Môi trường triển khai (Inteference)	CPU đa luồng thông thường (máy có GPU thì chạy sẽ nhanh và mượt hơn)
Thiết bị thu nhận ảnh	Webcam USB / Điện thoại (qua IP Webcam/DroidCam) / CameraIP

4.1.2. Công nghệ phần mềm

Thành phần	Công nghệ	Phiên bản	Vai trò
Ngôn ngữ	Python	3.10+	Ngôn ngữ chính
Xử lý ảnh	OpenCV	4.x	DIP preprocessing, đọc camera, vẽ annotation
Pose Estimation	Ultralytics YOLO	8.x	Phát hiện người + ước lượng keypoints
Deep Learning	PyTorch	2.x	Xây dựng và suy luận mạng PoseBiGRU
Tracking	ByteTrack	(built-in YOLO)	Theo dõi đối tượng đa khung hình
Giao diện	CustomTkinter	5.x	Xây dựng giao diện Desktop hiện đại
Đánh giá	Scikit-learn	1.x	Tính toán metrics: Accuracy, F1, Confusion Matrix

Thông báo	Telegram Bot API	-	Gửi cảnh báo qua http
Dữ liệu	Numpy, Pandas	-	Xử lý ma trận và dữ liệu bảng

4.2. Xây dựng giao diện người dùng (GUI)

4.2.1. Kiến trúc MVC đơn giản

Giao diện sử dụng pattern MVC (Model-View-Controller) đơn giản:

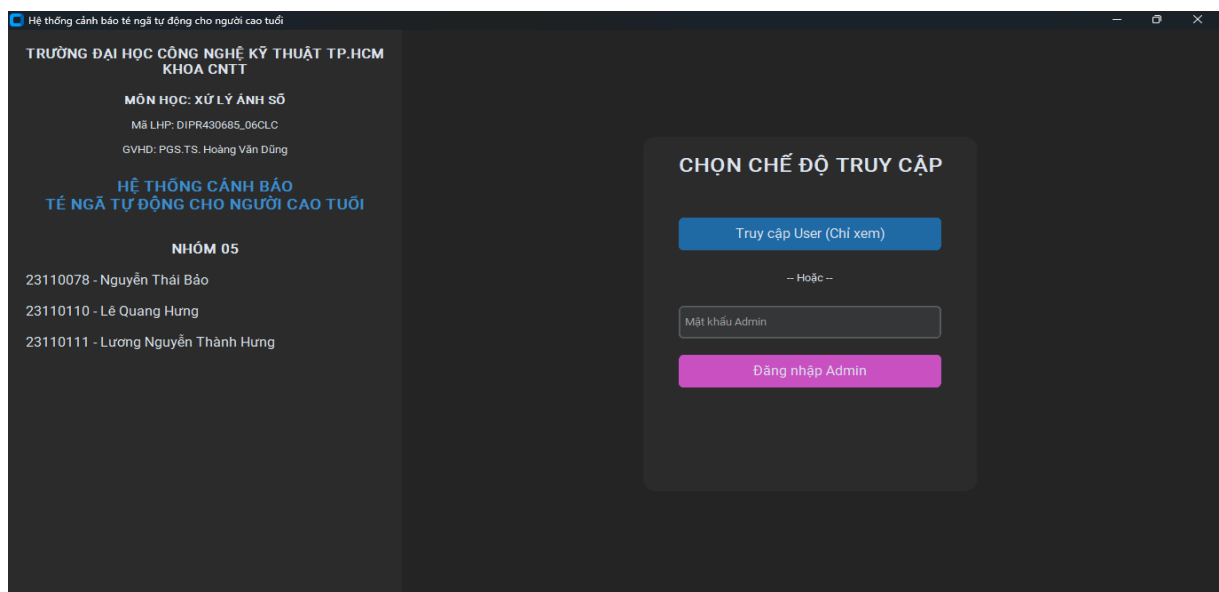
- Model: “CameraManager”, “InferenceManager” - xử lý dữ liệu
- View: Các class trong “ui/views/” - hiển thị giao diện
- Controller: “MainApp” - điều phối chuyển đổi giữa các màn hình

“MainApp” kế thừa “ctk.CTk”, quản lý một dictionary “self.views” chứa tất cả các view. Phương thức “show_view(name)” thực hiện: dùng camera ở view cũ → nâng view mới lên trên (tkraise) → bật camera nếu cần → cập nhật sidebar.

4.2.2. Các màn hình chính

Màn hình đăng nhập (LoginView):

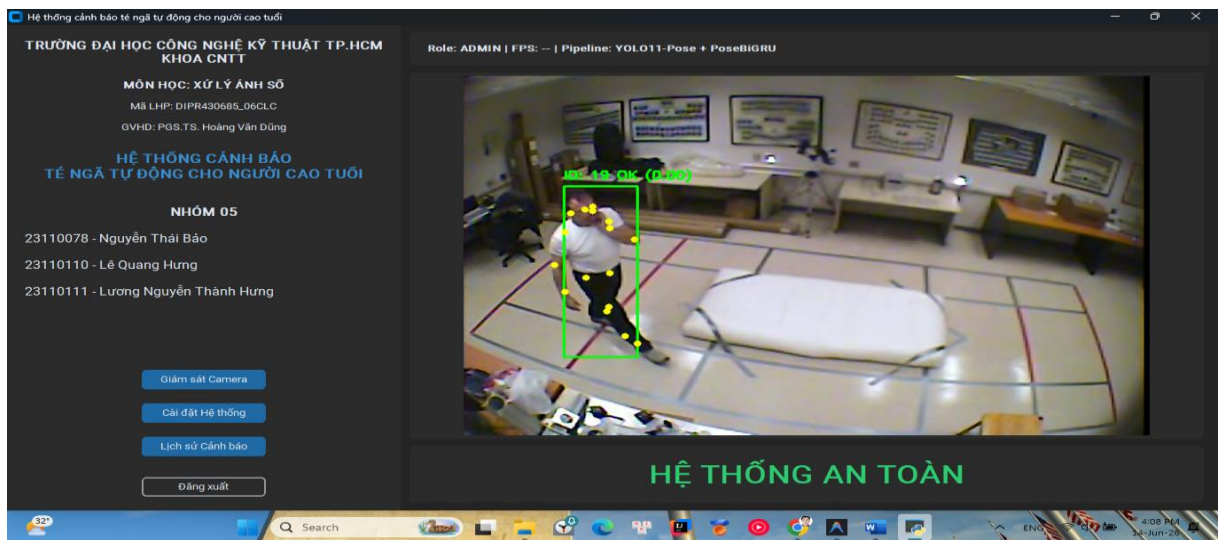
- Phân quyền 2 vai trò: Admin (có quyền truy cập Cài đặt) và Client (chỉ giám sát)
- Mật khẩu mặc định: “admin123” cho Admin
- Giao diện tối giản, tập trung vào 2 ô nhập liệu



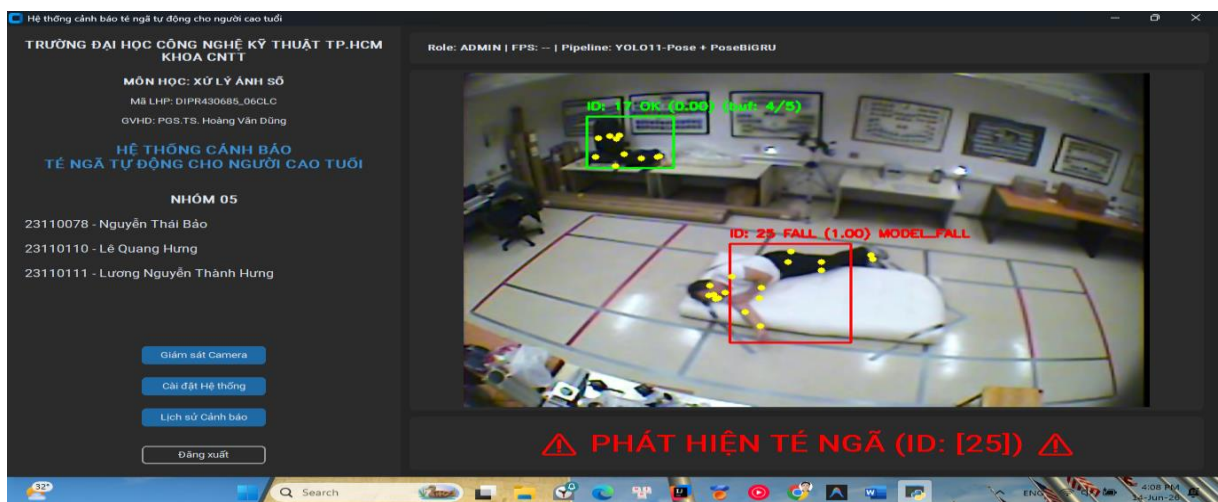
Hình 4. 1. Ảnh chụp màn hình Đăng nhập

Màn hình giám sát (AdminView / ClientView):

- Hiện thị video stream trực tiếp từ camera với annotation YOLO-Pose (bounding box, skeleton, Track ID)
- Dòng trạng thái lớn ở dưới: “HỆ THỐNG AN TOÀN” (xanh lá) hoặc “⚠ PHÁT HIỆN TẾ NGÃ (ID: X) ⚠” (đỏ)
- Khi phát hiện ngã: phát âm thanh “Beep(1000Hz, 500ms)” qua loa máy tính
- AdminView có toàn bộ chức năng; ClientView là phiên bản đơn giản hóa



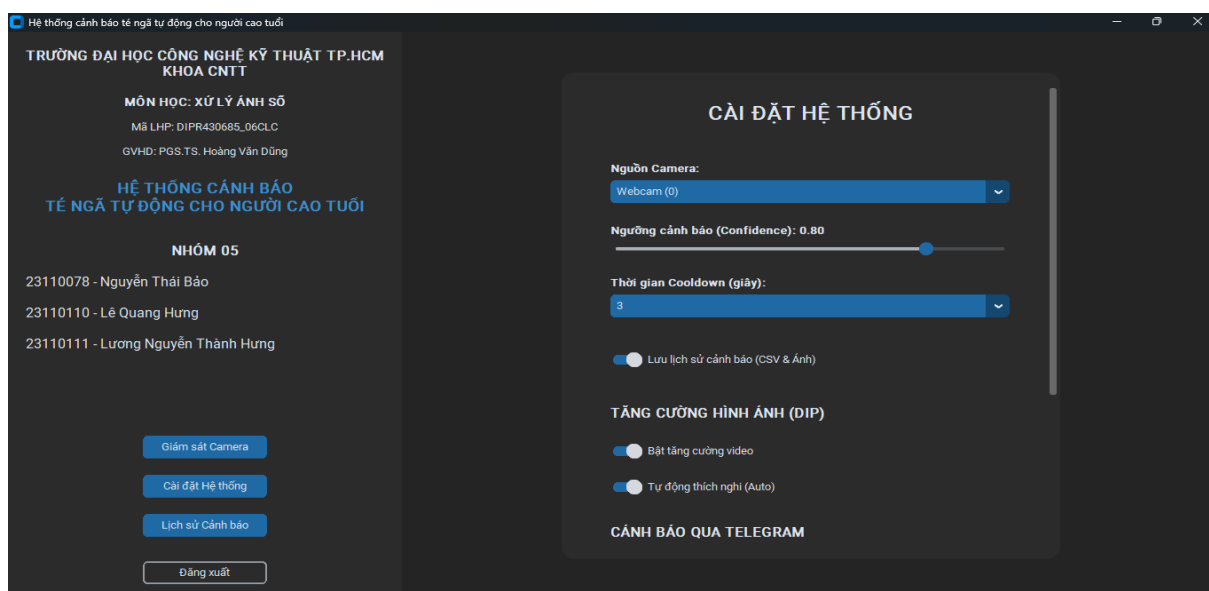
Hình 4. 2. Ảnh chụp màn hình Giám sát Camera khi hệ thống an toàn



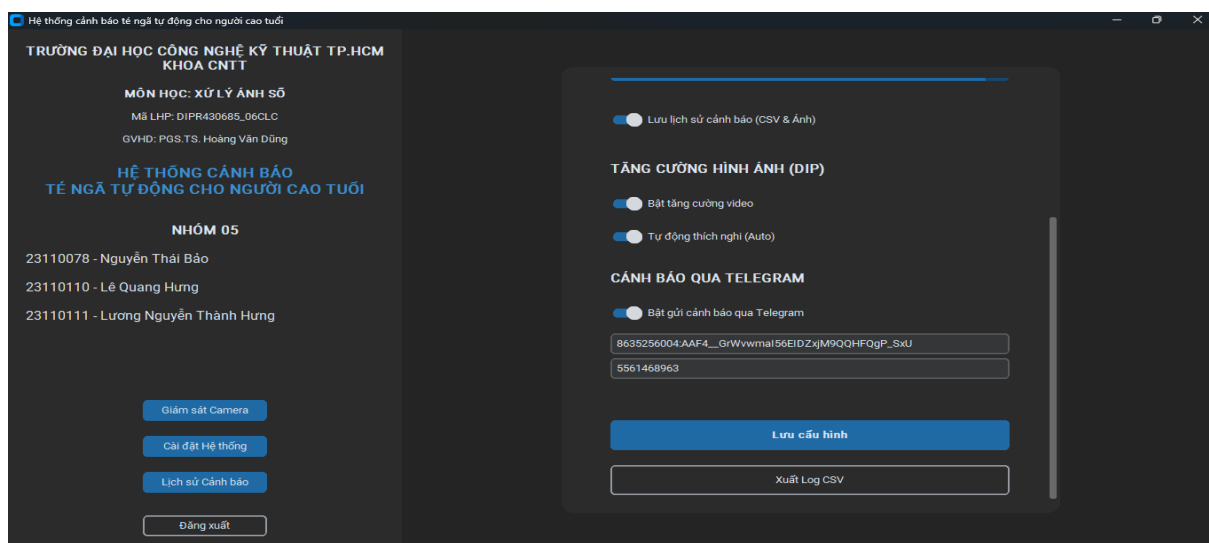
Hình 4. 3. Ảnh chụp màn hình Giám sát Camera khi phát hiện té ngã (khung đỏ)

Màn hình cài đặt hệ thống (AdminSettingsView):

- Nguồn Camera: Dropdown chọn Webcam, file video, hoặc nhập URL Camera IP (HTTP/RTSP)
- Ngưỡng cảnh báo (Confidence Threshold): Thanh trượt 0.0 – 1.0, điều chỉnh độ nhạy của PoseBiGRU
- Thời gian Cooldown: Dropdown 3/5/10/15 giây
- Module DIP: Bật/tắt tăng cường video, bật/tắt tự động thích nghi
- Telegram: Bật/tắt, nhập Bot Token và Chat ID
- Nút “Lưu cấu hình” và “Xuất Log CSV”



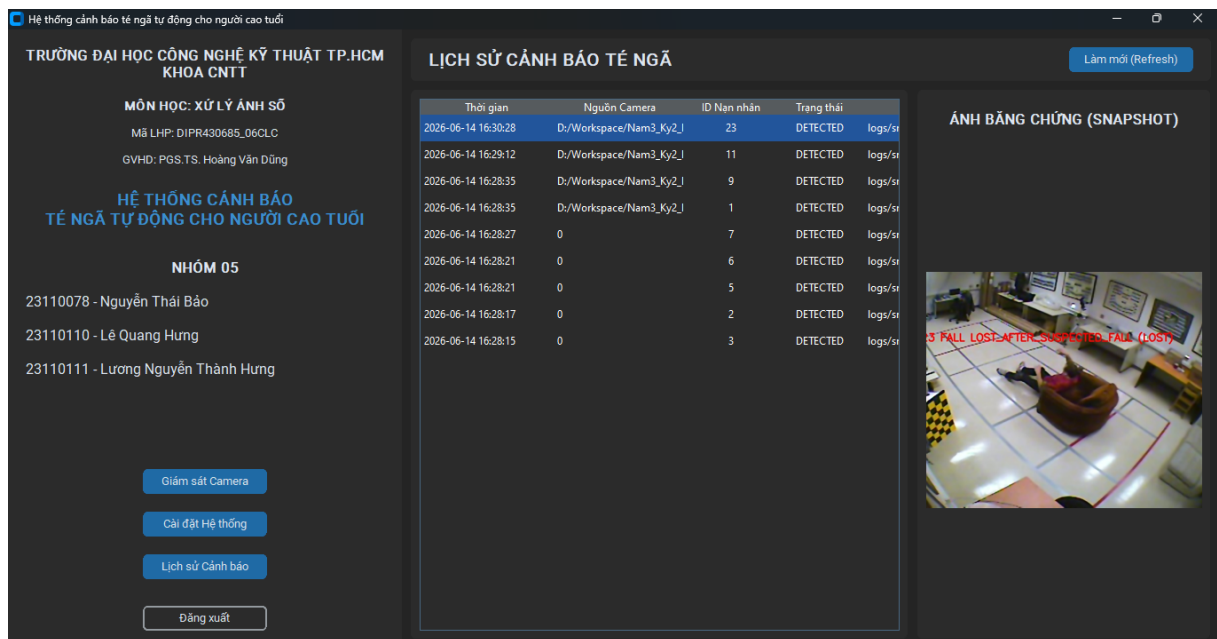
Hình 4. 4. Ảnh chụp màn hình Cài đặt hệ thống 1



Hình 4. 5. Ảnh chụp màn hình Cài đặt hệ thống 2

Màn hình lịch sử cảnh báo (HistoryView):

- Bảng dữ liệu (“tk.Treeview”) hiển thị toàn bộ lịch sử cảnh báo từ file CSV, sắp xếp mới nhất lên đầu
- 5 cột: Thời gian, Nguồn Camera, ID Nạn nhân, Trạng thái, Đường dẫn ảnh
- Bên phải: Khung xem trước ảnh Snapshot — click vào dòng lịch sử sẽ hiển thị ảnh bằng chứng tương ứng
- Nút “Làm mới” để reload dữ liệu



Hình 4. 6. Ảnh chụp màn hình Lịch sử cảnh báo té ngã với ảnh Snapshot

4.2.3. Thanh menu bên trái (Sidebar)

Sidebar cố định bên trái hiển thị:

- Tên trường, tên môn học, mã lớp học phần, giảng viên hướng dẫn, tên đề tài
- Danh sách thành viên nhóm
- 3 nút điều hướng: Giám sát Camera, Cài đặt Hệ thống, Lịch sử Cảnh báo
- Nút Đăng xuất

Các nút được ẩn/hiện tùy theo vai trò: Admin thấy cả 3 nút, Client chỉ thấy nút Giám sát và Lịch sử.

4.3. Huấn luyện mô hình PoseBiGRU

4.3.1. Bộ dữ liệu

Mô hình được huấn luyện trên bộ dữ liệu kết hợp từ nhiều nguồn:

Nguồn dữ liệu	Loại	Số lượng video	Mô tả
NTU RGB+D 60 (ntu60_hrnet.pkl)	Pre-trained	56,000+ clips	Bộ dữ liệu hành động quy mô lớn dùng để tiền huấn luyện (transfer learning)
UR Fall Detection Dataset	Chuẩn quốc tế	70 clips	30 clips ngã + 40 clips ADL, multi-camera
Multiple Cameras Fall Dataset	Chuẩn quốc tế	192 clips	24 kịch bản $\times$ 8 cameras, đa góc nhìn

4.3.2. Quy trình chuẩn bị dữ liệu (Data Pipeline)

Trích xuất skeleton: Chạy video qua YOLOv11s-Pose trên Kaggle GPU $\rightarrow$ xuất tọa độ 17 keypoints cho mỗi frame $\rightarrow$ lưu thành file “.pkl” (pickle) dạng MMAAction2 format.

Cắt chuỗi (Sliding Window): Chuỗi skeleton dài được cắt thành các clip có độ dài cố định 72 frames ($\approx$ 3 giây ở 24 FPS). Sử dụng stride = 1 cho maximum overlap.

Gán nhãn nhị phân: Mỗi clip được gán nhãn FALL (1) hoặc NON-FALL (0).

Tăng cường dữ liệu (Data Augmentation): Random temporal cropping, speed jittering (thay đổi tốc độ phát), random horizontal flip.

Cân bằng dữ liệu: Sử dụng “WeightedRandomSampler” với “fall_weight = 1.25” để bù đắp sự mất cân bằng giữa 2 lớp (clip Non-Fall thường nhiều hơn clip Fall).

4.3.3. Siêu tham số huấn luyện

Tham số	Giá trị
Optimizer	Adamw
Learning Rate	1e-3

Weight Decay	1e-4
Batch Size	128
Epochs	50
Sequence Length	72 frames
Hidden Size (GRU)	128
Num GRU Layers	2
Dropout	0.3
Gradient Clipping	1.0
Early Stopping Patience	8 epochs
Loss Function	CrossEntropyLoss (weighted)
LR Scheduler	ReduceLROnPlateau (factor=0.5, patience=3)

4.3.4. Môi trường huấn luyện

Quá trình huấn luyện được thực hiện trên Kaggle Notebook với GPU NVIDIA Tesla T4 x 2:

1. Upload toàn bộ dữ liệu skeleton (“.pkl”) lên Kaggle Dataset
2. Upload script `train.py` và `model.py` lên Kaggle Notebook
3. Chạy training với lệnh: `python train.py --dataset-source combined --epochs 50 --balanced-sampler`
4. Sau khi train xong, download file `best.pt` ($\approx 9\text{MB}$) về máy local đặt vào thư mục gốc của dự án

```
device: cuda
dataset source: combined
fps: 24.0
fall labels: [1, 2]
balanced sampler: True
train samples: 41099
val samples: 16666
train labels: [39918, 1181]
val labels: [16314, 352]
epoch 001 | train_loss=0.1720 | val_loss=0.0404 | thr=0.74 | precision=0.7307 | recall=0.9403 | fbeta=0.8457
epoch 002 | train_loss=0.0794 | val_loss=0.0371 | thr=0.80 | precision=0.7011 | recall=0.9062 | fbeta=0.8134
epoch 003 | train_loss=0.0633 | val_loss=0.0494 | thr=0.80 | precision=0.7477 | recall=0.9261 | fbeta=0.8472
epoch 004 | train_loss=0.0518 | val_loss=0.0317 | thr=0.80 | precision=0.7906 | recall=0.9119 | fbeta=0.8604
epoch 005 | train_loss=0.0481 | val_loss=0.0517 | thr=0.80 | precision=0.6875 | recall=0.9375 | fbeta=0.8210
epoch 006 | train_loss=0.0434 | val_loss=0.0358 | thr=0.80 | precision=0.7559 | recall=0.9148 | fbeta=0.8454
epoch 007 | train_loss=0.0388 | val_loss=0.0293 | thr=0.80 | precision=0.8249 | recall=0.8835 | fbeta=0.8597
epoch 008 | train_loss=0.0387 | val_loss=0.0274 | thr=0.75 | precision=0.7926 | recall=0.9119 | fbeta=0.8613
epoch 009 | train_loss=0.0359 | val_loss=0.0270 | thr=0.80 | precision=0.8128 | recall=0.9006 | fbeta=0.8642
```

Hình 4. 7. Ảnh chụp Kaggle Notebook đang train

4.4. Kết nối Camera IP từ điện thoại

Hệ thống hỗ trợ biến điện thoại thành Camera IP

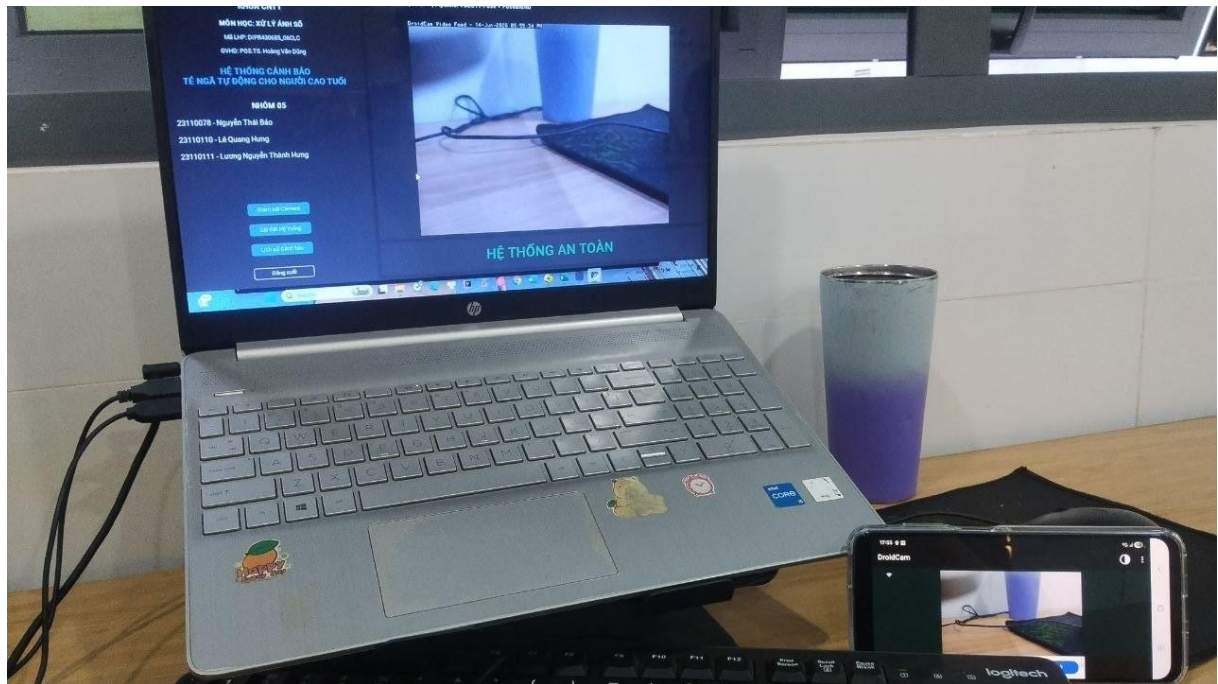
1. Cài ứng dụng IP Webcam (Android) hoặc DroidCam (Android/iOS) trên điện thoại

2. Kết nối cùng mạng WiFi với máy tính

3. Bật server trên ứng dụng → ghi nhận địa chỉ IP (ví dụ: “http://192.168.1.15:8080”)

4. Trên phần mềm: Cài đặt → Nguồn Camera → Nhập URL Camera IP → Nhập “http://192.168.1.15:8080/video” → Lưu cấu hình

OpenCV (“cv2.VideoCapture”) hỗ trợ sẵn đọc HTTP video stream và RTSP protocol, không cần cài thêm thư viện.



Hình 4. 8. Ảnh chụp điện thoại đang chạy IP Camera và Laptop nhận stream

PHẦN 5: THỬ NGHIỆM VÀ ĐÁNH GIÁ

5.1. Các độ đo đánh giá hiệu năng

5.1.1. Đánh giá năng lực phân loại

Dựa trên ma trận nhầm lẫn (Confusion Matrix), nhóm sử dụng các độ đo:

- Accuracy (Độ chính xác tổng thể): $\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}$

- Precision (Độ chính xác): $\text{Precision} = \frac{TP}{TP+FP}$

- Tỷ lệ cảnh báo đúng trong tổng số cảnh báo phát ra.

- Recall (Độ nhạy): $\text{Recall} = \frac{TP}{TP+FN}$ - Tỷ lệ phát hiện đúng trong tổng số ca té ngã thực tế. Đặc biệt quan trọng trong y tế đó là hạn chế bỏ sót sự cố

- F1-Score: $F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$ - Trung bình điều hòa, đánh giá cân bằng

Trong đó:

TP (True Positive): Hệ thống cảnh báo ngã và thực tế có ngã

TN (True Negative): Hệ thống không cảnh báo và thực tế không ngã

FP (False Positive): Hệ thống cảnh báo ngã và thực tế không ngã (báo động giả)

FN (False Negative): Hệ thống không cảnh báo nhưng thực tế có ngã (bỏ sót)

5.1.2. Đánh giá hiệu năng hệ thống

PS (Frames Per Second): Tổng thời gian xử lý pipeline (DIP → YOLO-Pose → PoseBiGRU → Heuristic) trên mỗi frame. Mục tiêu: ≥ 10 FPS trên CPU.

Độ trễ cảnh báo (Alert Latency): Thời gian từ khoảnh khắc bắt đầu ngã đến khi hệ thống phát cảnh báo. Mục tiêu: < 3 giây.

5.2. Kịch bản thử nghiệm

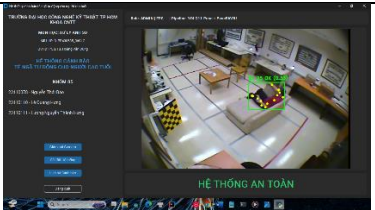
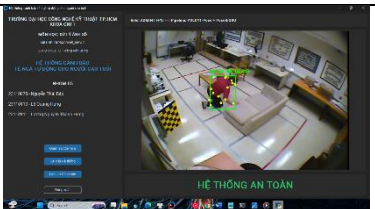
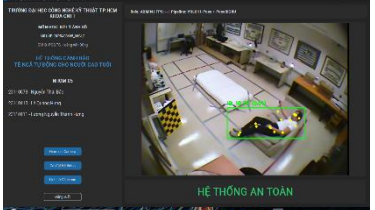
5.2.1. Kết quả Kịch bản Positive - Té ngã thực tế

Bảng dưới đây trình bày kết quả thực nghiệm trực tiếp trên hệ thống. Dữ liệu hình ảnh được hệ thống tự động chụp và lưu tại thư mục “logs/snapshots/” mỗi khi có cảnh báo.

ST T	Kịch bản	Mô tả	Kết quả kỳ vọng	Kết quả hệ thống	Minh họa (Ảnh chụp màn hình)
P1	Ngã chúi người về phía trước	Người đi bộ vấp chân ngã sấp mặt	DETECTE D	DETECT ED	
P2	Ngã ngửa	Trượt chân trên sàn nhà, ngã ngửa	DETECTE D	DETECT ED	
P3	Ngã khụy gối	Đứng rồi đột ngột gục xuống (mô phỏng choáng váng)	DETECTE D	DETECT ED	
P4	Ngã từ ghế	Đang ngồi ghế, trượt xuống sàn	DETECTE D	DETECT ED	

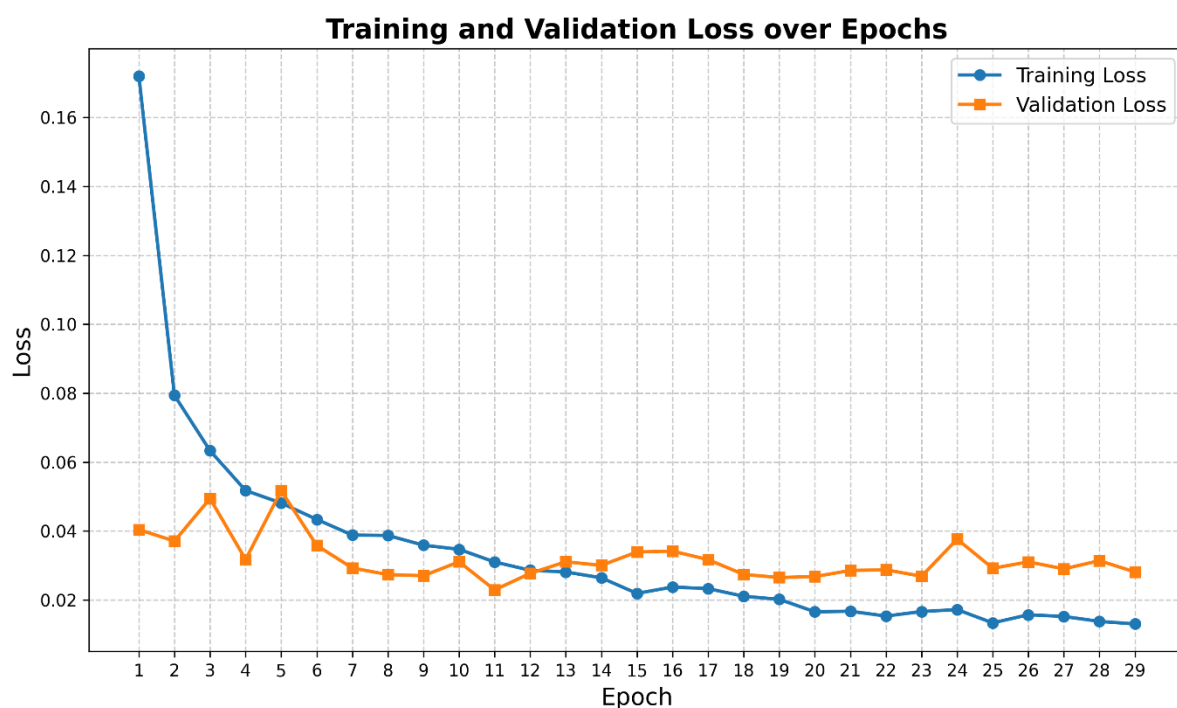
5.2.2. Kết quả Kịch bản Hard-Negative - Hành vi gây nhiễu

Kịch bản này cực kỳ quan trọng nhằm chứng minh hệ thống không bị báo động giả (False Positive) khi người dùng thực hiện các hoạt động sinh hoạt có quỹ đạo giống với việc té ngã.

ST T	Kịch bản	Mô tả	Kỳ vọng	Kết quả	Minh họa
N1	Cúi nhặt đồ	Cúi gập người nhặt vật dưới sàn	NOT DETECTED	NOT DETECTED	
N2	Ngồi xuống ghế / sofa	Ngồi thụp xuống ghế / sofa nhanh	NOT DETECTED	NOT DETECTED	
N3	Quỳ gối	Quỳ gối trên sàn để tìm đồ	NOT DETECTED	NOT DETECTED	
N4	Nằm có chủ ý	Chủ động nằm xuống giường/sofa	NOT DETECTED	NOT DETECTED	

5.3. Kết quả huấn luyện mô hình PoseBiGRU

5.3.1. Đường cong Loss



Hình 5. 1. Đường cong Training Loss và Validation Loss

Dựa vào đồ thị Loss, ta có thể rút ra một số kết luận quan trọng về quá trình học của mạng PoseBiGRU:

Về tốc độ hội tụ (Convergence) thì cả hai đường Training Loss và Validation Loss đều giảm mạnh và hội tụ rất nhanh ngay trong 10 epoch đầu tiên. Điều này minh chứng cho tính hiệu quả của chiến lược Học chuyển giao (Transfer Learning), khi mô hình tận dụng tốt bộ trọng số tiền huấn luyện từ tập dữ liệu NTU RGB+D.

Hiện tượng quá khớp (Overfitting): Validation Loss đạt mức cực tiểu (tối ưu nhất) ở khoảng epoch thứ 11 (~0.022). Sau giai đoạn này, Training Loss tiếp tục giảm dần về tiệm cận 0 (đạt ~0.013 ở epoch 29), trong khi Validation Loss có xu hướng dao động nhẹ và đi ngang (quanh mức 0.028 - 0.03). Dù có dấu hiệu quá khớp nhẹ ở các epoch cuối, sự chênh lệch là không đáng kể, mô hình vẫn giữ được khả năng tổng quát hóa rất tốt.

Mô hình tại epoch đạt Validation Loss thấp nhất đã được cơ chế “Early Stopping” lưu lại làm trọng số chính thức (“best.pt”) để sử dụng cho hệ thống suy luận thời gian thực, đảm bảo sự cân bằng giữa độ chính xác và khả năng tổng quát hóa trên dữ liệu thực tế.

5.3.2. Kết quả đánh giá trên tập Validation

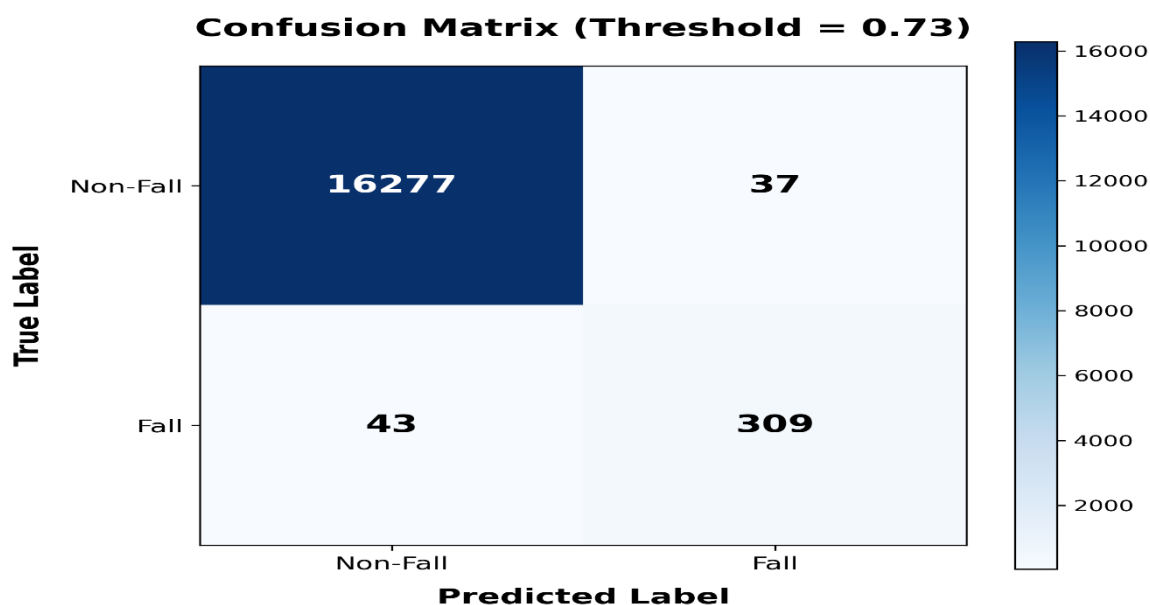
Dựa trên kết quả đánh giá ngưỡng (Threshold Analysis), dưới đây là bảng tổng hợp các chỉ số đánh giá của mô hình PoseBiGRU trên tập Validation (gồm 16.666 mẫu). Mô hình đạt mức cân bằng F1-Score tốt nhất tại ngưỡng 0.73.

Ngưỡng (Threshold)	Accuracy (%)	Precision (%)	Recall (%)	F1-Score(%)
0.3	99.42	84.04	89.77	86.81
0.5	99.45	85.56	89.20	87.34
0.60	99.46	86.39	88.35	87.36
0.73	99.52	89.31	87.78	88.54
0.80	99.51	89.97	86.65	88.28

Độ chính xác tổng thể (Accuracy): Rất cao (>99.4%), cho thấy hệ thống hoạt động vô cùng ổn định trên hầu hết các mẫu dữ liệu.

Tại ngưỡng tối ưu (0.73): Mô hình đạt F1-Score 88.54%, là sự cân bằng hoàn hảo giữa khả năng bắt trúng các ca ngã (Recall ~ 87.78%) và hạn chế báo động giả (Precision ~ 89.31%). Do đó, hệ thống thực tế (trong “inference_manager.py”) được thiết lập ngưỡng cảnh báo mặc định là “0.73” nhằm tối ưu hóa trải nghiệm người dùng.

5.3.3. Ma trận nhầm lẫn (Confusion Matrix)



Hình 5. 2. Ma trận nhầm lẫn tại ngưỡng tối ưu 0.73

True Positives (TP = 309): Mô hình nhận diện chính xác 309 tình huống ngã thực tế. Hệ thống phản ứng nhạy bén với các trường hợp nguy hiểm.

True Negatives (TN = 16277): Nhận diện chính xác tuyệt đối phần lớn các hoạt động sinh hoạt thường ngày (ADL) mà không kích hoạt cảnh báo nhầm, thể hiện tính ổn định cao của hệ thống.

False Positives (FP = 37): Chỉ có 37 ca báo động giả trên tổng số hơn 16.000 frame không ngã. Tỷ lệ này là vô cùng thấp, chứng tỏ PoseBiGRU đã học được các đặc trưng động lực học để phân biệt tốt giữa hành động ngã thật và các hành động dễ gây nhầm lẫn (cúi nhặt đồ, ngồi xổm gập người).

False Negatives (FN = 43): Có 43 ca ngã bị hệ thống bỏ sót. Đa số các tình huống này xảy ra khi người bị ngã ở quá xa camera, bị vật cản lớn che khuất phần lớn khung xương (occlusion), hoặc nằm lẫn vào các vật dụng có màu sắc tương đồng với trang phục. Đây là một điểm có thể cải thiện trong tương lai thông qua việc bổ sung đa góc nhìn (multi-camera view).

5.4. Phân tích và thảo luận

5.4.1. Điểm mạnh

Kiến trúc Two-tier Detection kết hợp Model AI (PoseBiGRU) và Heuristic cho phép phản ứng nhanh (heuristic) đồng thời đảm bảo độ chính xác cao (model). Khi model chưa đủ dữ liệu (ít frame), heuristic vẫn có thể phát hiện ngã ngay lập tức.

Module DIP thích nghi với auto-calibration giúp hệ thống hoạt động ổn định trong mọi điều kiện ánh sáng mà không cần người dùng can thiệp.

Cơ chế Debounce/Cooldown thông minh đảm bảo mỗi sự kiện ngã chỉ tạo ra DUY NHẤT 1 cảnh báo, tránh spam người giám hộ.

Mô hình PoseBiGRU nhẹ ($\approx 9\text{MB}$, $\sim 500\text{K}$ parameters) chạy được trên CPU thông thường, phù hợp triển khai tại gia đình không có GPU.

Hệ sinh thái cảnh báo đa kênh: Beep âm thanh (local) + Khung đỏ UI (local) + Telegram Bot (remote) + Log CSV + Ảnh Snapshot - đảm bảo không bỏ sót thông tin dù người giám hộ ở nơi khác.

5.4.2. Hạn chế và hướng cải thiện

YOLO mất dấu khi ngã vào sofa/nệm/chỗ khuất tầm nhìn: Khi người ngã vào đồ vật có màu tương đồng hoặc bị che khuất phần lớn cơ thể thì YOLO không phát hiện được person. Hướng cải thiện: fine-tune YOLO trên dữ liệu người nằm trên sofa.

False Positive khi đa người chồng chéo: Khi 2 người đứng sát nhau, ByteTrack có thể hoán đổi ID (ID switch), gây ra chuỗi keypoint bất thường khi đó thì kích hoạt false alarm. Hướng cải thiện: tích hợp ReID (Re-Identification) model.

Chưa hỗ trợ camera hồng ngoại (IR): Hệ thống hiện chỉ xử lý ảnh RGB. Camera hồng ngoại ban đêm có đặc tính ảnh khác (grayscale, contrast thấp), cần pipeline DIP riêng.

Thời gian thu thập đủ 30 frame: PoseBiGRU cần 30 frame (≈ 1.25 giây) để đưa ra dự đoán đầu tiên. Trong khoảng thời gian này, hệ thống phải dựa hoàn toàn vào heuristic nên có thể gây ra báo động giả khá nhiều. Hướng cải thiện: giảm “sequence_length” xuống 15-20 frame với mô hình retrained.

PHẦN 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết luận

Đề tài “**Xây dựng hệ thống cảnh báo té ngã tự động cho người cao tuổi**” đã hoàn thành các mục tiêu đề ra ban đầu:

Về mặt xử lý ảnh số (DIP): Hệ thống đã tích hợp thành công pipeline tiền xử lý thích nghi bao gồm khử nhiễu Gaussian, cân bằng sáng CLAHE, xử lý ngược sáng và cân bằng trắng. Module auto-calibration tự động đánh giá chất lượng khung hình đầu vào và chỉ kích hoạt các bộ lọc khi cần thiết, đảm bảo hiệu năng tối ưu.

Về mặt nhận dạng đối tượng: Mô hình YOLOv11s-Pose pre-trained kết hợp ByteTrack tracker đã thực hiện thành công bài toán phát hiện người, ước lượng tư thế 17 keypoints và theo dõi đa đối tượng xuyên suốt video stream.

Về mặt thuật toán phát hiện té ngã: Kiến trúc Two-tier Detection - kết hợp mạng PoseBiGRU-Attention (do nhóm tự thiết kế và huấn luyện) với hệ thống heuristic dựa trên cơ chế biểu quyết 8 chỉ số đã cho phép phân biệt chính xác giữa hành vi té ngã thật và các hoạt động sinh hoạt gây nhiễu (cúi nhặt đồ, quỳ gối, ngồi thụp xuống).

Về mặt hệ thống cảnh báo: Hệ thống đã triển khai đầy đủ chuỗi cảnh báo đa kênh: âm thanh local, hiển thị trực quan trên UI, ghi log CSV, lưu ảnh Snapshot bằng chứng, và gửi cảnh báo qua Telegram Bot kèm ảnh đến người giám hộ - tất cả hoạt động non-blocking trên các thread riêng biệt.

Về mặt giao diện: Ứng dụng Desktop hoàn chỉnh với 5 màn hình chức năng (Đăng nhập, Giám sát Camera, Cài đặt Hệ thống, Lịch sử Cảnh báo) được thiết kế trực quan, dễ sử dụng cho người cao tuổi và người giám hộ.

Đặc biệt, toàn bộ hệ thống có thể chạy trên CPU thông thường mà không cần GPU, với mô hình PoseBiGRU chỉ nặng khoảng 9MB — hoàn toàn phù hợp để triển khai tại gia đình với chi phí phần cứng tối thiểu.

6.2. Hướng phát triển

Nhóm đề xuất các hướng phát triển mở rộng để nâng cao hệ thống trong tương lai:

1. Hỗ trợ camera hồng ngoại (IR Night Vision): Mở rộng pipeline DIP để xử lý ảnh hồng ngoại, cho phép giám sát 24/7 kể cả khi tắt đèn hoàn toàn.
2. Multi-camera monitoring: Hỗ trợ đồng thời nhiều nguồn camera trên cùng một giao diện, hiển thị dạng lưới (grid), phù hợp cho viện dưỡng lão hoặc bệnh viện.
3. Tích hợp Edge AI: Triển khai hệ thống trên các thiết bị edge computing như NVIDIA Jetson Nano hoặc Raspberry Pi với mô hình quantized (INT8), tạo thành sản phẩm nhúng hoàn chỉnh.
4. Cải thiện Re-Identification: Tích hợp mô hình ReID để xử lý tốt hơn trường hợp đa người, tránh hoán đổi ID khi người đi ngang qua nhau.
5. Ứng dụng di động (Mobile App): Xây dựng ứng dụng giám sát trên điện thoại cho người giám hộ, tích hợp push notification thay cho Telegram, xem video stream từ xa.
6. Fine-tune YOLO trên dữ liệu người nằm/che khuất: Huấn luyện lại YOLO-Pose trên bộ dữ liệu bổ sung các trường hợp người nằm trên sofa, giường, sàn nhà, người ở trong các tư thế mà các mô hình yolo nhẹ hiện tại chưa detect người được giúp cải thiện khả năng phát hiện trong các tình huống che khuất.

References

- R. G. Stefanacci và J. R. Wilkinson, “Té ngã ở người cao tuổi,”
1] August 2025. [Trực tuyến]. Available: <https://www.msdmanuals.com/vi/professional/l%C3%A3o-khoa/t%C3%A9-ng%C3%A3-%E1%BB%9F-ng%C6%B0%E1%BB%9Di-caotu%E1%BB%95i/t%C3%A9-ng%C3%A3-%E1%BB%9F-ng%C6%B0%E1%BB%9Di-caotu%E1%BB%95i>. [Đã truy cập 05 06 2026].
- N. K. K. Võ, “Vì sao và nơi nào người cao tuổi dễ bị té ngã?,” 22 July
2] 2024. [Trực tuyến]. Available: <https://www.vinmec.com/vie/bai-viet/vi-sao-va-noi-nao-nguoi-cao-tuoi-de-bi-te-nga-vi>. [Đã truy cập 05 06 2026].
- A. P. Kaur, E. Nsugbe, A. Drahota, M. Oldfield, I. Mohagheghian and
3] R. A. Sporea, "State-of-the-art fall detection techniques with emphasis on floor-based systems—A review," *Biomedical Engineering Advances*, vol. 9, p. 100179, 2025.
- D. Hrubý, E. Hrubá and M. Černý, "Research of Fall Detection and
4] Fall Prevention Technologies: A Review," *Sensors*, vol. 26, p. 1192, 2026.
- K. Perliński, A. Faltyński and A. Ś. Swietlicka, "HumanFall
5] Detection with Infrared Imaging: A Comparison of Graph Convolutional Networks and YOLO," *sensors*, vol. 26, p. 2794, 2026.
- H. V. Dũng, "Chapter 1. Introduction".
6]
- H. V. Dũng, "Chapter 2. Image Enhancement in Spatial Domain".
7]